IT PORTFOLIO · BUILD HISTORY

Project Keystone Tracker

A GitHub-native project tracker with self-updating dashboards and an interactive website, built in one afternoon and demoed with a fictional 60-participant research study.

Builder	Tezana White	Date	October 10, 2026
Role	Product owner, requirements, PM, QA	Build time	~40 min to first release; v2 same afternoon
Repository	github.com/Tezana/project-keystone	Live site	tezana.github.io/project-keystone

01 Summary

Problem. Small project teams often keep plans in documents, with no single live view of phases, tasks, deadlines, and pending decisions, and nothing quick to check on a phone.

Solution. Use GitHub as the system of record: every task is an Issue, every phase is a Milestone, and two GitHub Actions automations keep a progress panel and a website current whenever a task changes. The website is fully interactive: open a task, mark it done, add notes, or create tasks, in a safe demo mode for visitors or an owner mode that writes to GitHub. Hosting is free and needs no server.

Demo scenario. To show it with realistic data, the tracker runs a fictional study: Lanternfield Labs validating a wearable hydration sensor with 60 participants, as 55 tasks across six phases.

55	6	2	$0
tasks loaded automatically	phases with due dates	automations (Actions)	hosting cost

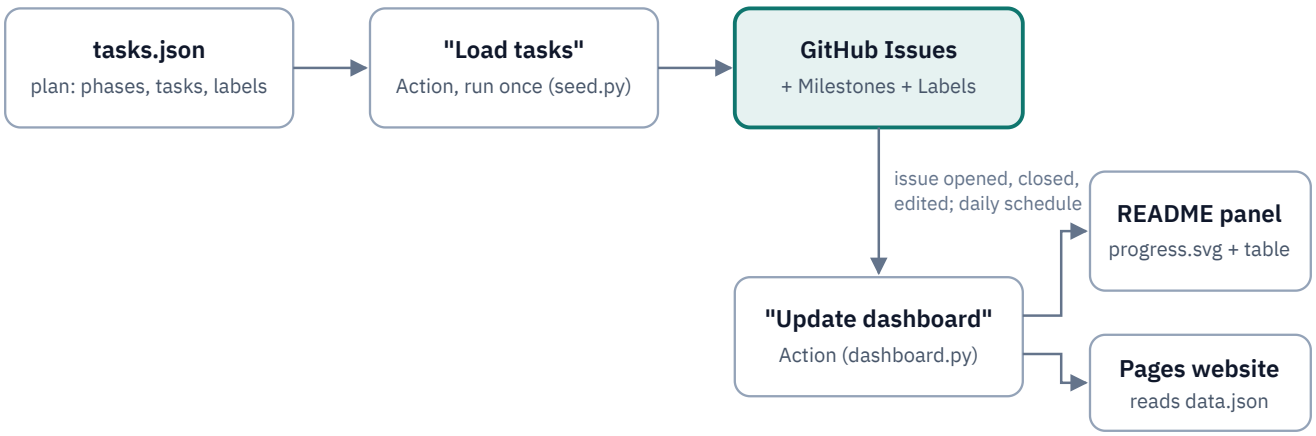
About 1,300 lines across Python, HTML/CSS/JavaScript, YAML, and JSON, with 37 automated browser checks. Built with an AI pair-programmer (Claude); requirements, design decisions, setup, and acceptance by Tezana White.

02 Requirements

Requirement	How it was met
Track the plan by phase	Six Milestones (Phase 0–5) with due dates; 55 Issues assigned to them
Hosted on GitHub	Repository, Issues, Actions, and Pages; nothing hosted elsewhere

See progress at a glance	Auto-drawn SVG progress panel and status table on the repo's front page
A usable visual view, like a portfolio site	Tracker website on GitHub Pages with phase cards, filters, search, and phone layout
Work inside the site, not on GitHub	Task pages, a New Task form, notes, and clickable summary boxes that open filtered lists
Let employers try it safely	Demo mode saves changes in the visitor's browser with a reset button; owner mode needs a private key
Realistic but safe demo data	Fictional research study and company; clean repository history; automated scan for leftover terms before publishing
Low effort to maintain	Close an Issue on GitHub; everything else updates itself within minutes

03 Architecture



The plan is loaded once; after that, GitHub Issues are the single source of truth. Any change to an Issue triggers the dashboard automation, which redraws the README panel and publishes fresh data for the website. In owner mode, the website also writes directly to Issues through the GitHub REST API.

Technology

Layer	Tools
Data and workflow	GitHub Issues, Milestones, Labels; plan stored as JSON
Automation	GitHub Actions (manual, event, scheduled, and chained `workflow_run` triggers); concurrency lock to prevent push conflicts
Scripts	Python 3.12, standard library only (GitHub REST API via `urllib`, pagination, rate-limit backoff, idempotent seeding)

Dashboard	Hand-generated SVG; README updated between marker comments
Website	Static HTML, CSS, vanilla JavaScript on GitHub Pages; responsive and dark-mode aware; accessible buttons, labels, focus states
Owner and demo modes	Owner: GitHub REST API from the browser with a fine-grained, repo-limited token. Demo: changes stored in the visitor's browser (localStorage)
Testing	Playwright browser tests (37 checks) including a simulated GitHub API; desktop and phone layouts; live deployment checks

04 Build timeline

October 10, 2026, Eastern Time. Commit and automation times from the repository history.

Time	Step	Detail
1:13 PM	Requirements	Track the launch plan by phase on GitHub; chose Issues plus a dashboard
2:17 PM	Scope change	Business GitHub sign-in was locked; moved to the personal account with a fictional company, codename, and neutral colors
2:19 PM	Built locally	Plan file (55 tasks), seed and dashboard scripts, two workflows, README; brand-name scan returned clean
2:24 PM	Repository created	New GitHub repository with initial README
2:28 PM	Access blocked	Push refused (HTTP 403); granted the app access to the new repo and refreshed the connection
2:31 PM	Pushed and seeded	Merged with GitHub's initial commit, pushed, started "Load tasks" remotely
2:33 PM	Tasks loaded	"Load tasks" succeeded in 2m 4s; chained "Update dashboard" ran in 9s
2:35 PM	Bug fix	"Up next" listed tasks newest-first; sorted by issue number
2:49 PM	Scope change	Repo confirmed public, which enables free Pages hosting; added a website requirement
2:51 PM	Website added	Tracker site plus `data.json` export; dashboard workflow extended
2:55 PM	Live	GitHub Pages enabled; site deployed and serving all 55 tasks

3:29 PM	Interactive version	Task pages, New Task form, notes, About page, demo and owner modes; 26 automated browser checks passed
3:39 PM	Clickable summary	Summary boxes open filtered task lists; 11 more checks passed
3:42 PM	Scope change	Demo data was too close to a real project; rebuilt as a fictional research study in a clean repository

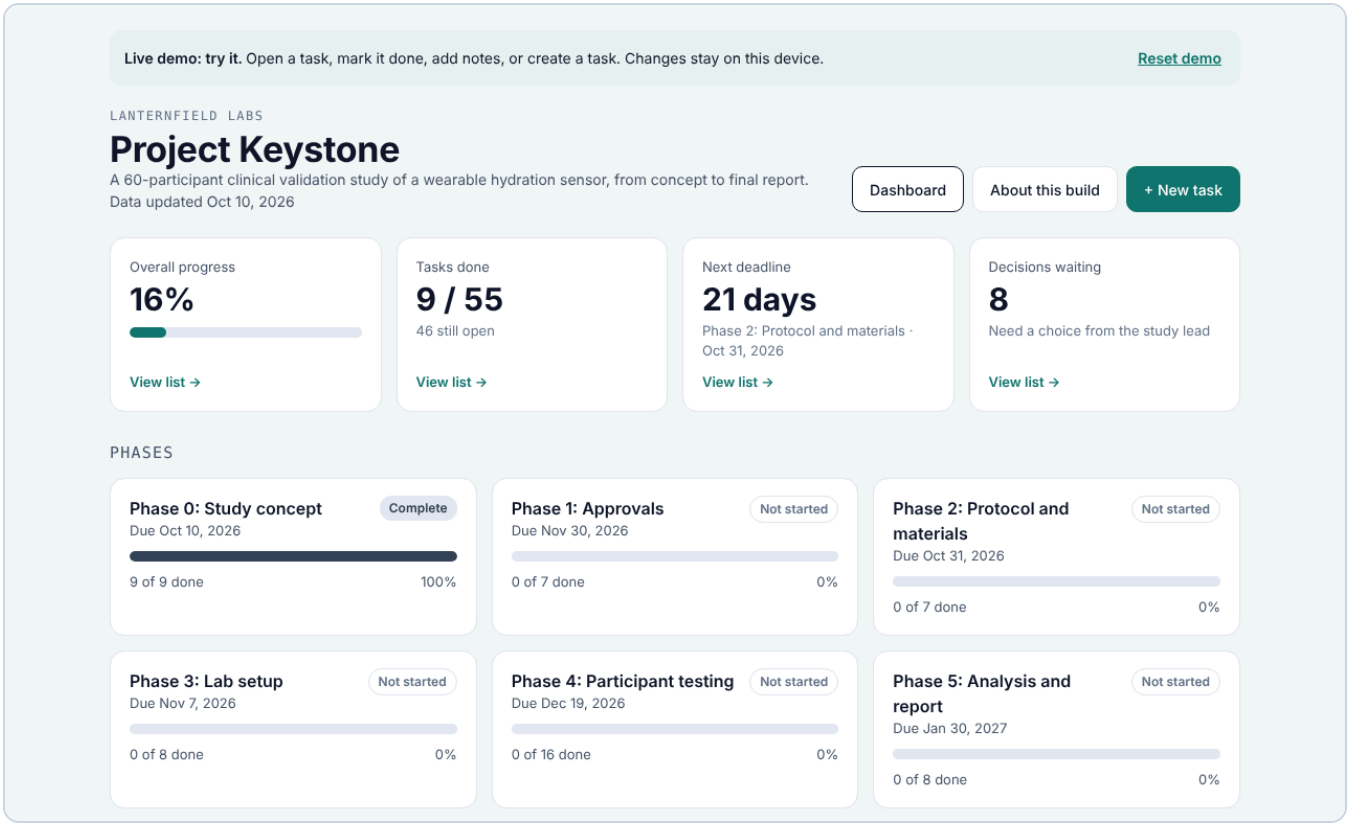
05 Challenges and resolutions

Challenge	Cause	Resolution
Couldn't create the planned GitHub organization	Too many sign-in attempts; temporary lockout	Built on the personal account instead
Push refused (HTTP 403)	The GitHub app only had access to previously selected repos	Added the new repo to the app's access list and re-linked the connection
Automation could create Issues, but the tool session couldn't	Session API limits	Moved seeding into a GitHub Action that uses the repo's own token, so setup is one click and repeatable
Hundreds of dashboard runs during seeding (risk)	Each new Issue fires an event	Actions-created Issues don't trigger workflows; a chained `workflow_run` redraws once after seeding
"Up next" in reverse order	GitHub API returns newest Issues first	Sort by Issue number; verified on the live README
Empty strip above the task list	CSS `display:flex` overrode the `hidden` attribute	Global `[hidden]{display:none}` rule; re-checked in screenshots
Pages couldn't be enabled from the tool	Settings API path not permitted	Owner enabled Pages in Settings (branch `main`, folder `/docs`)
Visitors shouldn't change real data	A public site can't safely hold a write key	Two modes: demo saves locally; owner mode needs a private key
Demo data too close to a real project	Editing Issues keeps old titles and file history visible on a public repo	Rebuilt with fictional data in a clean repository; scanned every file
Summary boxes misaligned	Buttons center their content vertically	Column layout with top alignment; re-checked in screenshots

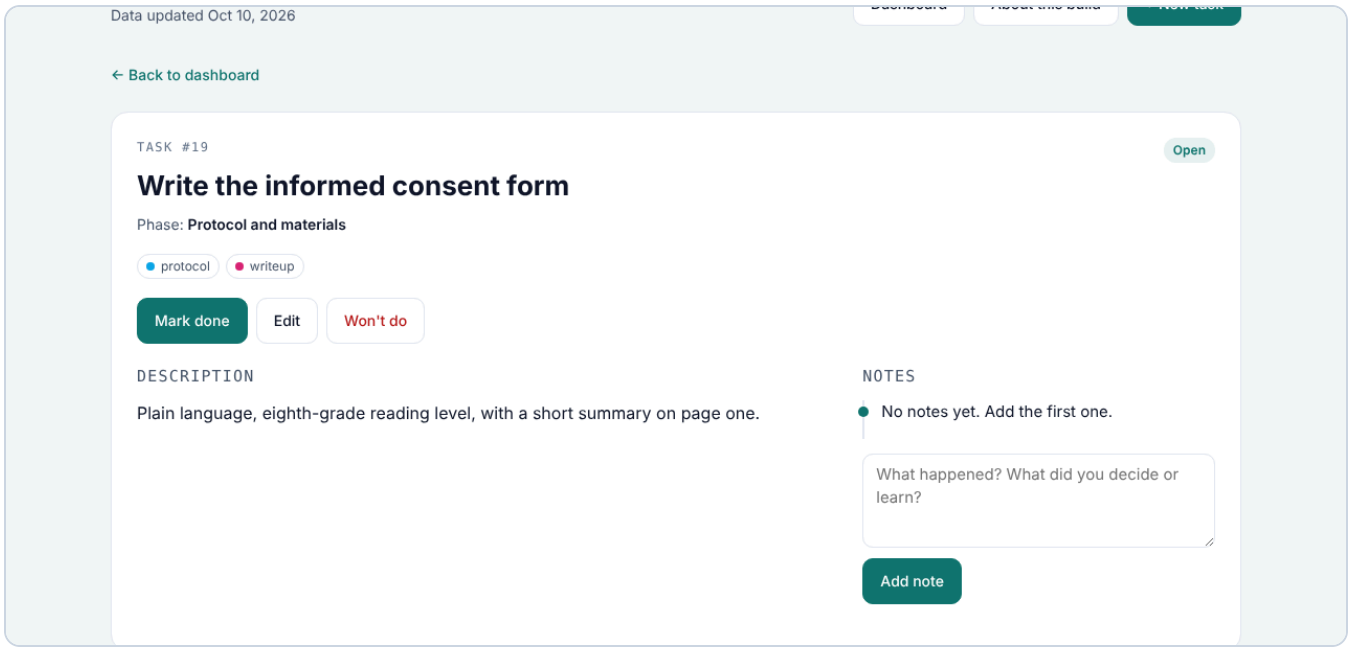
06 Testing and verification

- **Plan validation:** JSON parsed; task counts per phase checked (9, 7, 7, 8, 16, 8).
- **Local preview:** dashboard rendered from the plan file before any GitHub access; visual review of the SVG.
- **Confidentiality scan:** automated search of every file for real product, brand, and audience terms: no matches.
- **Live checks:** both workflows completed successfully; 55 Issues and 6 Milestones confirmed through the GitHub REST API.
- **Website:** headless browser screenshots at 1280 px and 390 px; no horizontal scroll, no script errors; live data confirmed after deploy.
- **Automated browser tests (37):** demo mode (open, mark done, edit, notes, new task, won't do, reset, never calls GitHub), owner mode against a simulated GitHub API (bad key rejected, PATCH, comments, new task in the right phase, lock), and the clickable summary boxes.

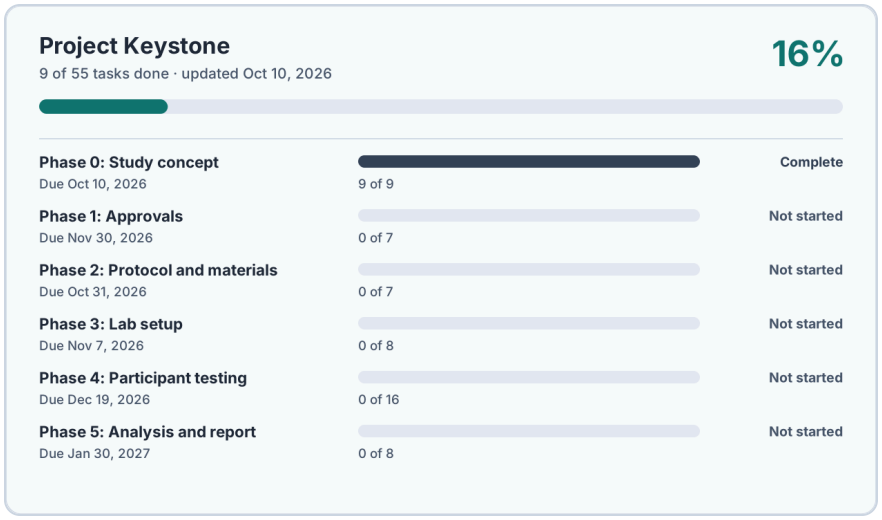
07 Screenshots



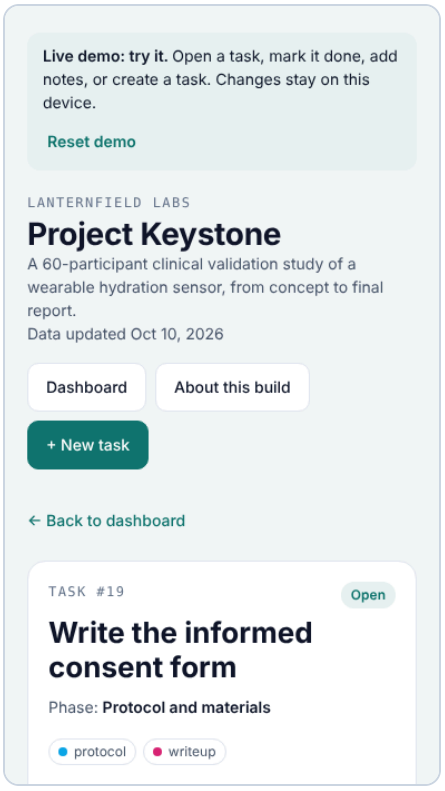
Dashboard: clickable summary boxes, phase cards, and the task list.



Task page: mark done, edit, or won't do, plus a dated notes log.



Auto-drawn progress panel on the repository's front page.



Phone layout.

08 Skills demonstrated

Project management	Technical
Work breakdown structure: 55 tasks in 6 phases with milestones and due dates	GitHub Actions: event, scheduled, manual, and chained triggers
Requirements gathering and two mid-build scope changes	Python REST API automation with pagination and backoff
Decision tracking (labeled and surfaced automatically)	Idempotent data seeding (safe to re-run)
Risk handling: access, lockout, and confidentiality	Static web app: responsive, accessible, dark mode
Status reporting with zero manual effort	Troubleshooting HTTP 403s, API limits, and CSS bugs
Stakeholder-driven iteration: three releases in one afternoon	Client-side REST API integration with token auth; Playwright test automation

09 Next steps

1. Add a burndown chart to the website as tasks close.
2. Replace the personal key with GitHub sign-in (OAuth) so a team can edit.
3. Package the tracker as a reusable template for any project plan.

Note: Lanternfield Labs, Project Keystone, and the hydration sensor study are fictional, created to demo the tracker with realistic data.